

Introduction to LangChain

LangChain is a powerful open-source framework designed for building applications powered by large language models (LLMs). Created by Harrison Chase and released in October 2022, LangChain provides a standardized interface for chains, agents, and memory systems that simplify the development of complex AI applications.

Core Components of LangChain:

1. **Models and Prompts:** LangChain abstracts away the differences between LLM providers (OpenAI, Anthropic, Google, etc.) behind a unified interface. The `ChatOpenAI` class, for example, wraps the OpenAI API and provides methods like `invoke()` and `stream()`. Prompt templates allow developers to create reusable, parameterized prompts with variables that are filled in at runtime.
2. **Chains (LCEL):** LangChain Expression Language (LCEL) is the modern way to compose components. Using the pipe operator (`|`), developers can chain together prompts, models, and output parsers into executable sequences. For example: `chain = prompt | llm | parser`. LCEL supports streaming, batch processing, and async execution out of the box.
3. **Document Loaders:** LangChain provides over 100 document loaders for ingesting data from various sources: PDFs (`PyPDFLoader`), web pages (`WebBaseLoader`), databases, APIs, and more. Each loader returns a list of Document objects with page content and metadata.
4. **Text Splitters:** After loading documents, text splitters break them into smaller chunks suitable for embedding and retrieval. The `RecursiveCharacterTextSplitter` is the most recommended, as it tries to split at natural boundaries (paragraphs, sentences) before falling back to character-level splitting.
5. **Embeddings:** LangChain integrates with embedding models from OpenAI, HuggingFace, Cohere, and others. The `OpenAIEmbeddings` class supports models like `text-embedding-3-small` (1536 dimensions) and `text-embedding-3-large` (3072 dimensions) for converting text into numerical vector representations.

Building RAG with LangChain

Retrieval-Augmented Generation (RAG) is the most popular application pattern built with LangChain. RAG combines the knowledge stored in a vector database with the generative capabilities of an LLM to answer questions grounded in specific documents.

The RAG Pipeline:

Step 1 - Indexing: Documents are loaded, split into chunks, converted to embeddings, and stored in a vector database like ChromaDB or PGVector. This is typically done once or on a schedule when documents are updated.

Step 2 - Retrieval: When a user asks a question, the question is embedded using the same embedding model. The vector database performs a similarity search to find the k most relevant document chunks. Typical values of k range from 3 to 10.

Step 3 - Generation: The retrieved chunks are formatted into a context string and combined with the user's question in a prompt template. The LLM generates an answer grounded in the provided context. Good prompts include instructions like 'only answer from the context' and 'cite your sources' to reduce hallucination.

Advanced RAG Techniques:

- Hybrid Search: Combining dense vector search with sparse keyword search (BM25) to improve retrieval quality, especially for queries with specific technical terms.
- Reranking: After initial retrieval, a cross-encoder model rescores the retrieved documents for more precise relevance ranking before passing to the LLM.
- Multi-Query Retrieval: Generating multiple reformulations of the user's question to cast a wider retrieval net and capture documents that might be missed by a single query.
- Self-Corrective RAG (Agentic RAG): Using LangGraph to build an agent that can evaluate its own answers, rewrite queries, and retry retrieval when the initial attempt produces low-quality results. This creates a feedback loop that significantly improves answer quality.

LangChain's modular design makes it easy to swap components: change the vector database, switch LLM providers, or add new retrieval strategies without rewriting the entire pipeline. This flexibility is key to building production-grade RAG systems.